



Résumé des Fonctions de la Librairie des Threads POSIX

Synthèse des fonctions de la bibliothèque POSIX Threads (pthread) [see a](#)

La bibliothèque POSIX Threads (pthread) fournit des fonctions pour la gestion des threads dans un environnement multithreadé. Voici une synthèse des principales fonctions, leur utilité et leurs prototypes :

1. pthread_create

- **Utilité** : Crée un nouveau thread.
- **Prototype** :

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *arg, void **retval);
```

- **Paramètres** :
 - `thread` : Pointeur vers l'identifiant du thread.
 - `attr` : Attributs du thread (NULL pour les valeurs par défaut).
 - `start_routine` : Fonction exécutée par le thread.
 - `arg` : Argument passé à la fonction du thread.
- **Valeur de retour** : 0 en cas de succès, un code d'erreur sinon.

2. pthread_exit

- **Utilité** : Termine le thread appelant.
- **Prototype** :

```
void pthread_exit(void *retval);
```

- **Paramètre** :
 - `retval` : Valeur de retour du thread (récupérable via `pthread_join`).
- **Remarque** : Un thread non joint par `pthread_join` devient zombie et peut causer une fuite mémoire.

3. pthread_join

- **Utilité** : Attend la fin d'un thread et récupère sa valeur de retour.

- **Prototype** :

```
int pthread_join(pthread_t thread, void **retval);
```

- **Paramètres** :

- `thread` : Identifiant du thread à joindre.
- `retval` : Pointeur vers la valeur de retour du thread (peut être NULL).

- **Valeur de retour** : 0 en cas de succès, un code d'erreur sinon.

4. pthread_detach

- **Utilité** : Détache un thread, libérant automatiquement ses ressources à sa terminaison.

- **Prototype** :

```
int pthread_detach(pthread_t thread);
```

- **Paramètre** :

- `thread` : Identifiant du thread à détacher.

- **Valeur de retour** : 0 en cas de succès, un code d'erreur sinon.

5. pthread_self

- **Utilité** : Retourne l'identifiant du thread appelant.
- **Prototype** :

```
pthread_t pthread_self(void);
```

- **Valeur de retour** : Identifiant du thread.

6. pthread_kill

- **Utilité** : Envoie un signal à un thread spécifique.
- **Prototype** :

```
int pthread_kill(pthread_t thread, int sig);
```

- **Paramètres** :
 - `thread` : Identifiant du thread cible.
 - `sig` : Signal à envoyer.
- **Valeur de retour** : 0 en cas de succès.

7. pthread_cancel

- **Utilité** : Demande l'annulation d'un thread.
- **Prototype** :

```
int pthread_cancel(pthread_t thread);
```

- **Paramètre** :
 - `thread` : Identifiant du thread à annuler.
- **Valeur de retour** : 0 en cas de succès.

8. pthread_mutex_lock / pthread_mutex_unlock

- **Utilité** : Gère l'accès concurrent aux ressources partagées via des mutex.
- **Prototypes** :

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- **Paramètre** :
 - `mutex` : Pointeur vers le mutex.
- **Valeur de retour** : 0 en cas de succès.
- **Paradigme** : l'exclusion mutuelle
Un seul thread à la fois peut accéder à une ressource critique.

9. pthread_cond_wait / pthread_cond_signal

- **Utilité** : Synchronisation conditionnelle entre threads.
- **Prototypes** :

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal(pthread_cond_t *cond);
```

- **Paramètres** :
 - `cond` : Condition à attendre ou signaler.
 - `mutex` : Mutex associé à la condition.
- **Valeur de retour** : 0 en cas de succès.
- **Paradigme**: la synchronisation conditionnelle.

Permet aux threads d'attendre une condition avant de continuer, optimisant ainsi l'ordre d'exécution.

10. pthread_key_create

- **Utilité** : Crée une clé pour stocker des données spécifiques à un thread.
- **Prototype** :

```
int pthread_key_create(pthread_key_t *key, void (*destructor)(void*));
```

- **Paramètres** :
 - `key` : Pointeur vers la clé à créer.
 - `destructor` : Fonction de nettoyage (optionnelle).
- **Valeur de retour** : 0 en cas de succès.

11. pthread_setspecific / pthread_getspecific

- **Utilité** : Associe ou récupère une valeur spécifique à une clé pour un thread.
- **Prototypes** :

```
int pthread_setspecific(pthread_key_t key, const void *value);  
void *pthread_getspecific(pthread_key_t key);
```

- **Paramètres** :
 - `key` : Clé à utiliser.
 - `value` : Valeur à associer (pour `pthread_setspecific`).
- **Valeur de retour** :
 - `pthread_setspecific` : 0 en cas de succès.
 - `pthread_getspecific` : Pointeur vers la valeur associée, ou NULL.